LEDSone Staff Dashboard — AI Assistant System

Complete Workflow & Setup Documentation

---

1. What Is the AI Assistant?

Every staff member's dashboard has a **floating AI button** (bottom-right corner). When clicked, it:

1. Reads ALL of that staff member's live requirement data from the database
2. Gives a **short task card** — max 5 priority tasks for today
3. On follow-up, gives a **full deep-dive** on whichever task the staff picks

This is not a generic chatbot. It is a **daily work prioritiser** — it knows the staff member's campaigns, products, spend, revenue, and alerts, and tells them exactly what to act on first.

Every day the task card is fresh. Because history is filtered to today only (`session_date = CURRENT_DATE`), every morning starts with a clean slate and the AI queries the latest live data. No manual reset needed.

---

2. The Two-Stage Conversation Flow

```
Staff opens dashboard
        │
        ▼
Clicks AI button (first time today)
        │
        ▼
Panel opens → checks today's DB history in background
        │
        ├─ History exists (reopened same day) → restores conversation
        │
        └─ No history → AI auto-loads (no typing needed)
               │
               ▼
┌──────────────────────────────────────────────────────┐
│  STAGE 1 — SHORT TASK CARD (auto, on open)           │
│                                                      │
│  "Good morning Jefri! Here are your 5 tasks for today:  │
│   1. Pause [LED Strip X] — €14 wasted, 0 conversions   │
│   2. [Klarna PMax] ROAS dropped 38% vs last period      │
│   3. [Transformer Y] OOS but still spending €8/day      │
│   4. Add negative keyword 'free led' — €6 wasted        │
│   5. [Shoparize PMax] top product — consider scaling     │
│   Which task would you like to start with?"             │
└──────────────────────────────────────────────────────┘
        │
        ▼
Staff types: "Tell me more about task 2"
        │
        ▼
┌──────────────────────────────────────────────────────┐
```

```
│  STAGE 2 — FULL DEEP-DIVE (on follow-up)                │
│                                                         │
│  AI gives: full analysis of that one task only          │
│  - Exact figures (cost, ROAS, comparison period)        │
│  - Why it's happening                                   │
│  - Recommended action steps                             │
└─────────────────────────────────────────────────────────┘
        │
        ▼
Staff continues the conversation for that task.
Next task → staff just types "ok next, tell me about task 3"
```

Key principle: The AI never dumps everything at once. Short card first — deep dive only on what the staff actually wants to work on.

---

3. Daily Fresh Task List — How It Works

Tasks reset automatically every day. No cron job. No manual action.

The chat history table filters by `session_date = CURRENT_DATE`:

```sql
SELECT role, content
FROM kamsi_ai_chat
WHERE session_date = CURRENT_DATE
ORDER BY id ASC;
```

- **New day** → query returns zero rows → widget triggers Stage 1 → AI fetches fresh live data → new task card
- **Same day, panel reopened** → query returns today's rows → conversation is restored exactly where it left off
- **Yesterday's rows** → still in the DB but never shown (date filter excludes them)

Tasks also change day-to-day because the underlying data changes:

- Campaign ROAS recalculates with yesterday's spend/revenue
- Wasteful products update as conversions roll in
- OOS status changes as stock levels change
- Top revenue products shift based on recent performance

If a problem is fixed (e.g. a product was paused), it won't appear in tomorrow's task card. If it persists (product still OOS, still spending), it keeps surfacing until resolved.

---

4. AI Model & API

Default Provider — Groq (10 staff)

Groq API — fast inference, free tier available

Groq Model Chain (tries in order until one responds)

```
1st choice:  qwen/qwen3-32b        ← primary (most capable)
2nd choice:  llama3-70b-8192       ← fallback
```

```
3rd choice:  llama-3.1-8b-instant  ← fallback
4th choice:  gemma2-9b-it          ← last resort
```

The model chain lives in **`lib/groq.js`** . `callGroqAI()` handles fallback silently — if the primary model fails or rate-limits, it tries the next one automatically.

`callGroqAI()` Function Signature

```
// lib/groq.js
const groqResult = await callGroqAI(messages);
// messages = [{ role: 'system', content: '...' }, { role: 'user', content: '...' }, ...]

// Returns:
// { ok: true,  text: 'AI response text' }
// { ok: false, error: 'error message' }
```

Premium Provider — NVIDIA NIM (Muguntha only)

NVIDIA Nemotron-3-Ultra-550B-A55B — 561B parameters, 1M context window, reasoning/thinking mode.

Used for Muguntha's management AI because her system prompt is large (7-day EOD + revenue + staff perf data). Lives in **`lib/nvidia.js`** .

```
// lib/nvidia.js
const result = await callNvidiaAI(messages, maxTokens);
// Falls back to callGroqAI() automatically if NVIDIA fails or key not set

// Strips <think>...</think> blocks from response before returning
```

NVIDIA NIM endpoint: `https://integrate.api.nvidia.com/v1/chat/completions` Model ID: `nvidia/nemotron-3-ultra-550b-a55b` Thinking mode: `chat_template_kwargs: { enable_thinking: true }`

Environment Variables Required

```
GROQ_API_KEY=gsk_...          ← all 11 staff (Groq chain)
NVIDIA_API_KEY=nvapi-...      ← Muguntha only (NVIDIA NIM, falls back to Groq if missing)
```

---

5. Chat History & Storage

Database

`SJ_CHAT_DB_URL` — separate Neon PostgreSQL database (not the main business DB)

Table Per Staff (auto-created on first use — no manual setup needed)

```
CREATE TABLE IF NOT EXISTS jefri_ai_chat (
  id           SERIAL PRIMARY KEY,
  session_date DATE        NOT NULL DEFAULT CURRENT_DATE,
```

```
  role         TEXT        NOT NULL,   -- 'user', 'assistant', or 'preference'
  content      TEXT        NOT NULL,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Role values:

- `user` — staff member's question
- `assistant` — AI response
- `preference` — daily learned pattern summary (written by preference learning system, see Section 22)

All 12 tables (11 staff + Muguntha): `kamsi_ai_chat`, `sukirtha_ai_chat`, `hetheesha_ai_chat`, `sonya_ai_chat`, `sajeepan_ai_chat`, `dilaksi_ai_chat`, `theekshy_ai_chat`, `thivajini_ai_chat`, `jefri_ai_chat`, `thasitha_ai_chat`, `mahima_ai_chat`, `muguntha_ai_chat`

Session Restore Behaviour — Correct Pattern (async/await sequential)

`aiLoadBrief` must **await the history check first** before firing any AI call. This prevents race conditions when the same user opens multiple browser tabs.

```
// CORRECT — history check first, AI call only if DB is empty
window.aiLoadBrief = async function() {
  // 1. Await history check
  const histData = await fetch('...ai-chat-history').then(r => r.json());
  if (histData.messages.length > 0) {
    // Restore from DB — no AI call needed
    renderHistory(histData.messages);
    return; // ← STOP HERE
  }
  // 2. DB empty — fire the AI call
  const data = await fetch('...ai-chat', { body: JSON.stringify({ history: [] }) }).then(r
=> r.json());
  renderBrief(data.message);
};

// WRONG — parallel approach causes multi-browser race condition
prefetchHistory();                    // async, not awaited
fetch('...ai-chat', { history: [] }) // fires simultaneously ← BUG
```

If history exists the widget shows:

> 💬 *Session restored. Ask me anything.*

This means the AI **never re-runs the daily brief** if the conversation was already started today — it continues from where it left off.

> ***Note:*** *Muguntha's widget uses the correct async/await pattern (fixed 2026-08-25). The other 10 staff widgets still use the older parallel approach — they work in single-browser usage but have a latent race condition in multi-browser scenarios. Backport pending.*

---

6. Full Request Flow (What Happens When AI Button Is Clicked)

```
Browser (staff dashboard)
        │
        │  1. Panel opens
        │  2. prefetchHistory() runs in background:
        │     GET /api/members-api?member=theekshy&type=ai-chat-history
        │     → loads today's chat from SJ_CHAT_DB_URL
        │
        │  3. If no history today → auto-sends brief request:
        │     POST /api/members-api?member=theekshy&type=ai-chat
        │     body: { message: '', history: [] }
        │
        │  3b. If history exists → restores chat, skips brief
        │
        ▼
members-api.js / requirement.js (Vercel Serverless Function)
        │
        │  4. Opens pg.Pool to DATABASE_URL (business DB)
        │  5. Runs up to 5 parallel queries (Promise.all):
        │     - Campaign ROAS vs prior 30-day period
        │     - Top 8 revenue products
        │     - Wasteful products (0 conv, spend > €3)
        │     - Out-of-stock still spending
        │     - Shopify orders (for relevant market)
        │  6. Closes pool (await db.end())
        │  7. Builds system prompt with live data rows
        │  8. Calls callGroqAI(messages)
        │
        ▼
Groq API (qwen/qwen3-32b → fallback chain)
        │
        │  9. Returns AI response text
        │
        ▼
API handler
        │
        │  10. Returns { ok: true, message: '...', is_daily_brief: true/false }
        │
        ▼
Browser
        │
        │  11. Renders task card / response in chat panel
        │  12. POST .../ai-chat-save → saves AI response to SJ_CHAT_DB_URL
        │
        ▼
Staff reads task card → picks a task → types follow-up
        │
        │  13. Repeat steps 3-12 with actual message + last 6 messages of history
        │      (history is passed in request body — last 6 messages = 3 exchanges)
```

Timeout: If AI takes longer than 90 seconds, the widget shows:

> 💍 ***I went down a rabbit hole and lost track of time!*** *Click ↻ Refresh and I'll be quicker this time, promise!* 🧋

---

7. System Prompt Structure

This is what the AI sees for every request. Data sections are filled with **live DB rows at query time**.

```
You are [Name]'s Google Ads AI at LEDSone [market]. Data: [from] to [to].

CAMPAIGNS (name|cost|rev(prev)|ROAS|conv):
Pmax | Jeff | Klarna | NEWALL | cost:€1842|rev:€6200(prev:€9100)|ROAS:336%|conv:24
Pmax | Jeff | Shoparize | IT | cost:€420|rev:€980(prev:€1400)|ROAS:233%|conv:6
...

TOP 8 REVENUE PRODUCTS (title|rev|cost|conv):
#1 LED Strip 5m 24V Warm White|€1240rev|€310cost|8conv
#2 Transformer 60W|€840rev|€190cost|5conv
...

WASTEFUL PRODUCTS — 0 conversions, spending (title|cost):
LED Panel 120x30 Cool White|€18wasted
Dimmer Switch 4CH|€11wasted
...

OOS STILL SPENDING (title|cost):
LED Neon Flex 5m|€22spend
Smart Driver 100W|€9spend
...

ROAS TARGET: 300%+ for all campaigns.

INSTRUCTIONS:
- OPENING (empty message): Reply with ONLY a short task card:
  "Good morning [Name]! Here are your X tasks for today:
  1. [one-line task — key data point]
  2. ...
  Which task would you like to start with?"
  Max 5 tasks. No analysis. No extra text.
- FOLLOW-UP (user asks about a task): Give full analysis +
  action steps for that task only.
- Format: "1. [ACTION] — [campaign/product] ([reason with € or % figure])"
```

---

8. Staff Coverage & Data Sources

Staff	Market	API File	Route	Live Data
Kamsi	ledsone.co.uk	members-api.js	`?member=kamsi&type=ai-chat`	GSC impressions/clicks, missing meta title+desc counts, product priority (high demand + low organic)

Sukirtha	ledsone.co.uk	members-api.js	`?member=sukirtha&type=ai-chat`	GSC top pages, keyword overlap, organic opportunity
Hetheesha	ledsone.co.uk	members-api.js	`?member=hetheesha&type=ai-chat`	Fix tracker progress, conversion tracking status
Sonya	ledsone.co.uk	members-api.js	`?member=sonya&type=ai-chat`	PMax ROAS, wasteful search terms, top converters
Sajeepan	ledsone.co.uk	members-api.js	`?member=sajeepan&type=ai-chat`	Campaign performance, keyword automation
Dilaksi	ledsone.co.uk	requirement.js	`?fn=dilaksi-ai-chat`	SEO: low-engagement pages, high-priority products
Theekshy	ledsone.de	members-api.js	`?member=theekshy&type=ai-chat`	DE campaigns ROAS + prev period, top/wasteful products, negative KW candidates, OOS spending
Thivajini	ledsone.fr	members-api.js	`?member=thivajini&type=ai-chat`	FR campaigns ROAS + prev period, hero products, wasteful spend, OOS, Shopify FR orders
Jefri	ledsone.de + IT	requirement.js	`?fn=jefri-ai-chat`	DE+IT campaigns ROAS + prev period, top/wasteful products, OOS, Shopify DE orders
Thasitha	ledsone.de	requirement.js	`?fn=thasitha-ai-chat`	DE campaigns dynamic (`group_name='Thasi'`), ROAS, wasteful products, OOS
Mahima	ledsone.de	requirement.js	`?fn=mahima-ai-chat`	DE campaigns ROAS + prev period, top/wasteful products, OOS, Shopify DE orders

---

9. Campaign ID Configuration

Staff	How Campaigns Are Identified
Theekshy	Hard-coded array in members-api.js: `TH_CAMPAIGNS = [23714290257, 23684837882]`
Thivajini	Hard-coded array in members-api.js: `TV_CAMPAIGNS = [23103582865, 23533025729, 23405519670]`
Jefri	Hard-coded array in requirement.js: `_JEFRI_AI_IDS = ['23141810147', '23411228109', '22539594891', '23473840779', '23340277562']`

Thasitha	**Dynamic** — queried at runtime: `SELECT campaign_id FROM google_ads.campaigns WHERE group_name = 'Thasi'` — automatically picks up new campaigns
Mahima	Hard-coded array in requirement.js: `_MAHIMA_AI_IDS = ['20763699505', '23684789991', '23053104908', '23431543574', '23926509987']`

---

10. Database Connections Used

Connection	Purpose
`DATABASE_URL`	Business data — Google Ads performance, merchant products, Shopify orders
`SJ_CHAT_DB_URL`	Chat history storage (all 11 staff AI chat tables)

Critical: Always Use `pg.Pool`, Never `pg.Client` in AI Handlers

AI handlers run 5 queries in parallel with `Promise.all`. `pg.Client` only supports one query at a time — using it with `Promise.all` crashes with:

> *`Error: client is already executing a query`*

```
// CORRECT — Pool supports concurrent queries
const db = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: { rejectUnauthorized: false },
  max: 3,
  connectionTimeoutMillis: 15000,
  statement_timeout: 55000,
});
const [{ rows: r1 }, { rows: r2 }, { rows: r3 }] = await Promise.all([
  db.query(...), db.query(...), db.query(...),
]);
await db.end(); // always close after use

// WRONG — crashes on parallel queries
const c = new Client(...);
await c.connect();
await Promise.all([c.query(...), c.query(...)]); // ← crash
```

`Pool` Scope in `requirement.js`

`requirement.js` has many nested IIFEs that each declare their own `const { Pool } = require('pg')`. The AI handlers sit at the module's top level — they need `Pool` available at that scope.

Fix already applied: Line 7 of `requirement.js` now reads:

```
const { Client, Pool } = require('pg');
```

This makes `Pool` available everywhere in the file. Do not add `const { Pool } = require('pg')` inside the AI handler functions in `requirement.js` — it is already declared at the top.

---

11. API Routes Reference (All AI Endpoints)

members-api.js (`/api/members-api`)

Member	Chat	History	Save
kamsi	`?member=kamsi&type=ai-chat`	`&type=ai-chat-history`	`&type=ai-chat-save`
sukirtha	`?member=sukirtha&type=ai-chat`	`&type=ai-chat-history`	`&type=ai-chat-save`
hetheesha	`?member=hetheesha&type=ai-chat`	`&type=ai-chat-history`	`&type=ai-chat-save`
sonya	`?member=sonya&type=ai-chat`	`&type=ai-chat-history`	`&type=ai-chat-save`
sajeepan	`?member=sajeepan&type=ai-chat`	`&type=ai-chat-history`	`&type=ai-chat-save`
theekshy	`?member=theekshy&type=ai-chat`	`&type=ai-chat-history`	`&type=ai-chat-save`
thivajini	`?member=thivajini&type=ai-chat`	`&type=ai-chat-history`	`&type=ai-chat-save`

requirement.js (`/api/requirement`)

Staff	Chat	History	Save
dilaksi	`?fn=dilaksi-ai-chat`	`?fn=dilaksi-chat-history`	`?fn=dilaksi-chat-save`
jefri	`?fn=jefri-ai-chat`	`?fn=jefri-chat-history`	`?fn=jefri-chat-save`
thasitha	`?fn=thasitha-ai-chat`	`?fn=thasitha-chat-history`	`?fn=thasitha-chat-save`
mahima	`?fn=mahima-ai-chat`	`?fn=mahima-chat-history`	`?fn=mahima-chat-save`

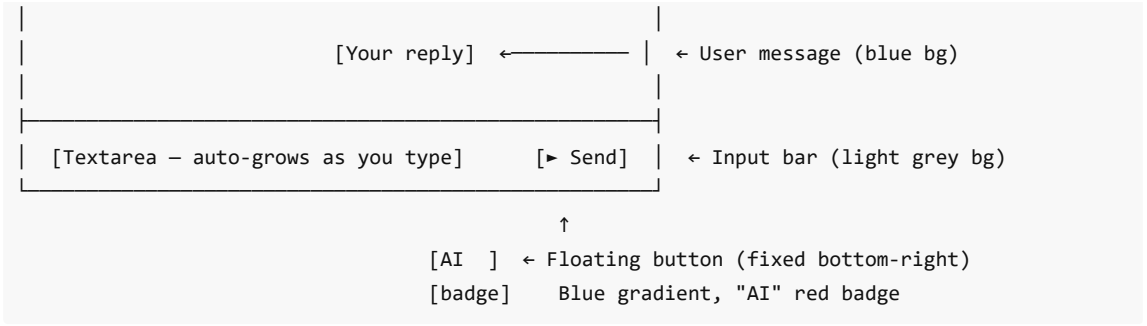
> ***Note:*** *Clear endpoints exist in the API (* `ai-chat-clear` */* `chat-clear` *) but the UI no longer shows a Clear button. The Refresh button re-runs* `aiLoadBrief()` *which resets the in-memory history and fetches a new task card — but today's DB rows remain (the session restore will pick them up if the page is refreshed).*

---

12. Widget UI — Unified Design (All Staff)

All 11 staff dashboards use the **same UI** — Sajeepan's blue style. No per-staff colours.

```
┌──────────────────────────────────────────────────┐
│  😈  [Name]'s AI Assistant          [↻ Refresh] [✕] │  ← Blue gradient header
├──────────────────────────────────────────────────┤
│                                                  │
│  👋 Good morning, [Name]!                        │
│     Fetching your live data…                     │  ← Bot message (light blue bg)
│                                                  │
│  • • •  (thinking animation)                     │
│                                                  │
│  1. Pause LED Strip X — €14 wasted, 0 conv       │  ← Task card
│  2. Klarna PMax ROAS dropped 38%                 │
│  3. ...                                          │
│  Which task would you like to start with?        │
```



Colour Specification (same for all staff)

Element	Colour
Button + header gradient	`#1a73e8` → `#0d47a1`
Bot message background	`#f0f4ff`
User message background	`#1a73e8`
Bot message text accent (bold)	`#0d47a1`
Send button	`#1a73e8` (hover: `#0d47a1`)
Send button disabled	`#b0c4e8`
Input focus border	`#1a73e8`
Badge	`#ea4335` (red)
Input bar background	`#fafbfc`

Behaviour Details

Behaviour	Detail
First open today	Auto-triggers task card (no typing needed)
Reopen same day	Restores conversation from DB — shows "Session restored"
Refresh button	Resets in-memory state and re-fetches a new task card
Thinking animation	3-dot pulse shown while AI is loading
Textarea	Auto-grows as you type (max 80px height)
Enter key	Sends message (Shift+Enter for new line)
Timeout	90 seconds — shows retry prompt if exceeded

---

13. How to Add an AI Assistant for a New Staff Member

Step 1 — Chat infrastructure (in the API file)

```
async function getNameChatClient() {
  const c = new Client({
    connectionString: process.env.SJ_CHAT_DB_URL,
    ssl: { rejectUnauthorized: false },
    connectionTimeoutMillis: 10000
  });
  await c.connect();
  await c.query(`CREATE TABLE IF NOT EXISTS name_ai_chat (
    id SERIAL PRIMARY KEY,
    session_date DATE NOT NULL DEFAULT CURRENT_DATE,
    role TEXT NOT NULL,
    content TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
  )`);
  return c;
}
async function handleNameChatHistory(req, res) {
  let c; try { c = await getNameChatClient();
    const { rows } = await c.query(`SELECT role, content FROM name_ai_chat WHERE
session_date = CURRENT_DATE ORDER BY id ASC`);
    return res.status(200).json({ ok: true, messages: rows });
  } catch(e) { return res.status(500).json({ ok: false, error: e.message });
  } finally { if(c) await c.end().catch(()=>{}); }
}
async function handleNameChatSave(req, res) { /* same pattern */ }
```

Step 2 — AI handler (use Pool, not Client)

```
async function handleNameAiChat(req, res) {
  const body = req.body || {};
  const userMessage = (body.message || '').trim();
  const history = Array.isArray(body.history) ? body.history : [];
  try {
    // In members-api.js: Pool is available from top-level require
    // In requirement.js: Pool is declared at line 7 — do NOT re-declare here
    const db = new Pool({ connectionString: process.env.DATABASE_URL,
      ssl: { rejectUnauthorized: false }, max: 3,
      connectionTimeoutMillis: 15000, statement_timeout: 55000 });

    const [{ rows: campRows }, { rows: wasteRows }] = await Promise.all([
      db.query(`SELECT ... FROM google_ads.campaign_performance WHERE campaign_id=ANY($1)`,
[IDS]),
      db.query(`...`),
    ]);
    await db.end();

    const systemPrompt = `You are [Name]'s AI...
INSTRUCTIONS:
- OPENING (empty message): Short task card only, max 5 tasks.
- FOLLOW-UP: Full analysis + action steps for that task only.`;
```

```
    const messages = [
      { role: 'system', content: systemPrompt },
      ...history,
      { role: 'user', content: userMessage || 'Assign my tasks for today. Short task card
only.' }
    ];
    const groqResult = await callGroqAI(messages);
    if (!groqResult.ok) return res.status(502).json({ ok: false, error: groqResult.error });
    return res.status(200).json({ ok: true, message: groqResult.text, is_daily_brief:
!userMessage });
  } catch(e) { return res.status(500).json({ ok: false, error: e.message }); }
}
```

Step 3 — Add routing

members-api.js — inside the member's `if (member === 'name')` block:

```
if (type === 'ai-chat-history') return handleNameChatHistory(req, res);
if (type === 'ai-chat-save')    return handleNameChatSave(req, res);
if (type === 'ai-chat')         return handleNameAiChat(req, res);
```

requirement.js — inside `module.exports` :

```
if (fn === 'name-ai-chat')      return handleNameAiChat(req, res);
if (fn === 'name-chat-history') return handleNameChatHistory(req, res);
if (fn === 'name-chat-save')    return handleNameChatSave(req, res);
```

Step 4 — Add the HTML widget

Copy the widget block from any current staff page (e.g. `sajeepan.html` or `theekshy.html` — all are now identical in structure). Change only:

- The comment header (`[NAME] AI ASSISTANT WIDGET`)
- `[Name]'s AI Assistant` label text
- The subtitle text
- The 3 API URLs (`api_chat` , `api_hist` , `api_save`)
- The hint text at the bottom of the brief

Paste before `</body>` . Do **not** change colours — all staff share the same blue style.

---

14. Known Gaps (What the AI Does NOT See)

These data sources are live/external-only and cannot be summarised in the AI prompt:

Staff	Missing Data	Reason
Kamsi	Req 3 (GA4), Req 6 (Duplicate/Price check)	Shopify live API — not stored in DB
Sukirtha	Req 2 (Low CTR ledsone.de), Req 3 (GA4)	ledsone.de GSC not in DB; GA4 is live API

Hetheesha	Req 3, 4, 5	Shopify live API only
Dilaksi	Req 4 (Content Gap)	Per-keyword Semrush live lookup — no DB summary
Muguntha	UK Shopify refunds	Shopify Admin API only — not in DB
Muguntha	Germany Sales Decline detail	Static snapshot — not queryable for AI context
Muguntha	Staff ID perf for Theekshy/Thivajini/Hetheesha/Sukirtha	No entries in `data/staff-ids.js` for these members
Piranav	All	Admin dashboard — AI not built yet (backlog)

---

15. Bugs Fixed During Build

Bug	Root Cause	Fix
Kamsi/Sukirtha AI panel wouldn't open (click did nothing)	A Python code-generation script wrote `\n\n` as a **literal newline character** inside a JS regex literal — `/\n\n[newline]/g` spans two lines, which is a JS SyntaxError. The entire `<script>` block failed silently, so `addEventListener('click', aiToggle)` never ran.	Rewrote the full widget `<script>` block cleanly in both files so `formatAiText` uses proper escape sequences: `.replace(/\n\n/g,'  ').replace(/\n/g,' ')`
Kamsi AI panel earlier crash (separate issue)	`'Clear today's chat history?'` — unescaped apostrophe in single-quoted JS string crashed the widget script	Changed to double quotes: `"Clear today's chat history?"`
Kamsi AI handler crashed on load	`pg.Client` used with `Promise.all` (6 simultaneous queries) → `"client is already executing a query"`	Switched to `pg.Pool` with `max: 3`
Jefri/Thasitha/Mahima: `Pool is not defined`	`Pool` was only declared inside nested IIFEs in `requirement.js`, not at module top level	Added `Pool` to the top-level `require('pg')` on line 7: `const { Client, Pool } = require('pg')`
Sajeepan duplicate widget	Added a second AI widget without checking one already existed	Removed duplicate immediately
Muguntha: multi-browser — question shows, no response	`prefetchHistory()` and brief AI call ran in parallel. Browser B started a 10-second NVIDIA call even when DB history existed. Its `.finally` reset `isLoading=false` mid-send, corrupting the send lock — next question fired but response couldn't write back cleanly	Rewrote `aiLoadBrief` as `async function` — awaits history check first, returns immediately if DB has messages, only fires AI call when DB is empty

Muguntha: AI answered "I don't have yesterday's data"	EOD fetch only checked today — no historical data passed to AI	Extended to 7-day range: 14 staff × 7 days = 98 parallel GitHub API calls in one `Promise.all`

---

16. Vercel Deployment & Function Limit

How to Deploy

Just push to `main` — Vercel auto-deploys on every push:

```
git add .
git commit -m "your message"
git push
```

Vercel builds and deploys automatically. No CLI command needed.

Critical: 12 Serverless Function Limit (Hobby Plan)

Vercel Hobby plan allows a maximum of **12 serverless functions** — each file in the `api/` folder counts as one function.

This project already uses:

File	Count
`api/members-api.js`	1
`api/requirement.js`	1
`api/lib/groq.js`	(shared lib, not a function)
+ other existing api files	...

Rule: Never create a new file in `api/` for AI handlers. All new AI handlers must go inside either `members-api.js` (preferred for staff on the members dashboard) or `requirement.js` (for others). Creating a new file would push the project over the function limit and break deployment.

---

17. Kamsi Special Data Fields

Kamsi's AI handler receives two extra fields that no other staff member has. These are passed in every fetch request body from `kamsi.html`:

```
body: JSON.stringify({
  history: [],
  metaSummary: window._kamsiMeta || null,       // meta title/desc audit summary
  prioritySummary: window._kamsiPriority || null // product priority list
})
```

These are populated earlier in the page by the normal Kamsi dashboard data load. The AI handler reads them from `req.body` and includes them in the system prompt if present. If you ever rewrite or copy the Kamsi widget, make

sure these two fields are kept in the fetch body — without them the AI loses context about which products are missing meta data.

---

18. Environment Variables Required

Variable	Used For
`GROQ_API_KEY`	Groq AI model API calls (`lib/groq.js`) — all 11 staff
`NVIDIA_API_KEY`	NVIDIA NIM API (`lib/nvidia.js`) — Muguntha only, falls back to Groq if missing
`SJ_CHAT_DB_URL`	Chat history storage — all 11+ staff `*_ai_chat` tables
`DATABASE_URL`	Business data — Google Ads, orders, GSC, merchant products, listings
`FEED_TRACKER_DB_URL` / `AUTH_DATABASE_URL`	Requirement trackers — hetheesha_fix_tracker, feed_optimization_tracker, jefri_req6_tracker
`EOD_GITHUB_TOKEN`	GitHub API PAT — reads EOD report files from `digitalmarketing69140951-sys/eod-reports`

---

19. Muguntha Management AI — Architecture

Muguntha is a **full admin / team manager**, not a campaign manager. Her AI is built differently from all other staff assistants.

Data Fetched Per Request (one parallel batch)

```
Promise.all([
  // Tier 1 — People & Operations
  14 staff × 7 days = 98 GitHub API calls (EOD submissions),
  AUTH DB: hetheesha_fix_tracker (Req1),
  AUTH DB: hetheesha_fix_tracker_r2 (Req2),
  AUTH DB: feed_optimization_tracker (Sajeepan Req4),
  AUTH DB: jefri_req6_tracker (Jefri Req6),

  // Tier 2 — Business Health
  DATABASE_URL: UK revenue today/yday/7d-avg/30d-avg (sub_source_id=233),
  DATABASE_URL: DE revenue today/yday/7d-avg/30d-avg (sub_source_id=108),
  DATABASE_URL: 2026 New Listings count (listings.shopify_listings, tag=2026New),

  // Tier 3 — Staff Accountability
  DATABASE_URL: Kamsi staff ID perf (30d revenue + dead stock),
  DATABASE_URL: Dilaksi staff ID perf,
  DATABASE_URL: Sajeepan staff ID perf,
  DATABASE_URL: Jakshan staff ID perf,
  DATABASE_URL: Sonya staff ID perf,
  DATABASE_URL: Mahima staff ID perf,
])
```

~104 parallel I/O calls resolved before the AI call.

EOD GitHub Path

```
GET https://api.github.com/repos/digitalmarketing69140951-sys/eod-reports/contents/eods/{Name}/{YYYY-MM-DD}.md
Authorization: Bearer {EOD_GITHUB_TOKEN}
HTTP 200 = submitted | HTTP 404 = not submitted
```

14 staff names: Kuberan, Piranav, Mahima, Sonya, Kamsi, Hetheesha, Dilaksi, Sukirtha, Theekshy, Thivagini, Jefri, Sajeepan, Jakshan, Thasitha

DB Connections Used

Connection	What
`DATABASE_URL`	Business data (orders, listings, staff ID perf)
`FEED_TRACKER_DB_URL` || `AUTH_DATABASE_URL`	Requirement tracker tables
`SJ_CHAT_DB_URL`	Chat history (`muguntha_ai_chat` table)

Routes (all in `api/muguntha.js` via `?action=` routing)

Action	Purpose
`?action=ai-chat`	Main AI call (brief or follow-up)
`?action=ai-chat-history`	Fetch today's chat history
`?action=ai-chat-save`	Save one message to DB
`?action=ai-chat-clear`	Clear today's session

---

20. Multi-Browser Race Condition — Fix Pattern

Problem (latent in all 11 widgets, fixed in Muguntha's on 2026-08-25):

When a staff member opens the same dashboard in two browser tabs:

1. Both call `aiLoadBrief()` simultaneously
2. Both fire `prefetchHistory()` + AI brief call in parallel
3. Browser B's AI brief call (10 seconds) finishes → `.finally: isLoading = false`
4. This resets the lock while Browser B's `aiSend` is still in flight
5. Result: question message appears, thinking spinner disappears, no bot response shown

Fix — async/await sequential pattern:

```
// CORRECT (Muguntha's widget)
window.aiLoadBrief = async function() {
  if (isLoading) return;
  isLoading = true;
```

```
  // Step 1: await history check
  try {
    const histData = await fetch('...ai-chat-history').then(r => r.json());
    if (histData.ok && histData.messages.length > 0) {
      renderHistory(histData.messages);
      briefLoaded = true; isLoading = false; sendBtn.disabled = false;
      return; // ← bail out — no AI call needed
    }
  } catch (e) {}

  // Step 2: only reaches here if DB is empty — fire AI call
  try {
    const data = await fetch('...ai-chat', { body: '{"history":[]}' }).then(r => r.json());
    renderBrief(data.message);
    briefLoaded = true;
  } catch (e) {
    showErrorMessage();
  }
  isLoading = false;
};
```

Backport status: Only Muguntha's widget uses this pattern. The other 10 staff widgets still use `prefetchHistory()` parallel approach — safe for single-browser but should be updated.

---

21. Prompt Intelligence Upgrade — All 12 AI Assistants (2026-08-25)

All 12 system prompts (11 staff + Muguntha) were upgraded with the following structure additions:

URGENCY RANKING

Each staff AI now has a numbered priority order tailored to their role. Added before INSTRUCTIONS in every system prompt.

Staff	#1 Urgency Signal
Sajeepan, Theekshy, Thivajini, Jefri, Thasitha, Mahima	OOS products still spending budget
Sonya	ROAS drop vs prior period
Hetheesha	High-revenue product + open tracker fix + low GSC impressions
Sukirtha	Products dropped to zero sales this month
Kamsi	Top-revenue product appearing in declining pages
Dilaksi	High-traffic pages with engagement rate below 20%
Muguntha	Staff missing EOD 2+ days in a row

BEHAVIOUR RULES

Added to every system prompt:

- Never say "I don't have that data" — all data is in the prompt, use it
- Never hallucinate numbers — only use exact figures from the data above
- Never use filler phrases ("Great question!", "Certainly!", "Of course!")
- Always name the specific person/campaign/product/metric — never be vague
- Keep responses short and direct

OPENING FORMAT — Emoji Priority Flags

Task card now uses emoji flags:

- 🔴 Critical/High urgency
- 🟡 Medium urgency
- 🟢 Low urgency

FOLLOW-UP FORMAT

Every follow-up response now ends with **one clear action** the staff member can take immediately.

Time-Aware Greeting

Replaced hardcoded "Good morning" across all 12 AI assistants:

```
// Client-side (HTML widget loading message)
var _hr = new Date().getHours();
var _greet = _hr < 12 ? 'Good morning' : _hr < 17 ? 'Good afternoon' : 'Good evening';

// Server-side (API system prompt)
const _hr = new Date().getHours();
const greeting = _hr < 12 ? 'Good morning' : _hr < 17 ? 'Good afternoon' : 'Good evening';
```

Applied to: 12 HTML pages + `members-api.js` (6 prompts) + `requirement.js` (4 prompts) + `muguntha.js` (1 prompt).

---

22. Preference Learning System — All 12 AI Assistants (2026-08-25)

What It Does

Every day, when a staff member opens the dashboard for the first time (new day detected), the AI:

1. Reads yesterday's `role='user'` messages from the chat table
2. Sends them to an AI model for pattern analysis
3. Saves the result as a `role='preference'` row in the same table
4. Fetches the most recent preference row
5. Injects it into today's system prompt as a `LEARNED FROM PREVIOUS SESSIONS` block

The AI learns what each staff member always drills into and personalises the task card accordingly. Over time, the opening brief gets shorter because the AI pre-surfaces what the staff member would ask anyway.

DB Storage — No New Table

Reuses existing `*_ai_chat` tables. A new `role` value is added:

Role	Meaning
`user`	Staff member's question (existing)
`assistant`	AI response (existing)
`preference`	Daily learned pattern summary (new)

```
-- Example preference row saved after analysis
INSERT INTO kamsi_ai_chat (session_date, role, content)
VALUES ('2026-08-25', 'preference',
  '- Kamsi always drills into declining pages first — lead with full click detail
   - She always asks about quick win pages — include in brief
   - She focuses on specific page handles, not general summaries');
```

Shared Helper Functions

Added to `members-api.js` and `requirement.js`:

```
// Analyses yesterday's user messages and saves preference
async function analyseAndSavePreference(pool, tableName)

// Fetches the most recent preference row
async function getLatestPreference(pool, tableName)
```

Added to `muguntha.js` (standalone, uses NVIDIA):

```
async function analyseMuguntha Preference(client)
async function getMuguntha Preference(client)
```

Injection Into System Prompt

```
// Before INSTRUCTIONS block in every system prompt:
${learnedPreference ? `LEARNED FROM PREVIOUS SESSIONS (use this to personalise today's task
card):\n${learnedPreference}\n\n` : ''}INSTRUCTIONS:
```

Analysis Model

Staff	Model
All 11 staff	Groq `qwen/qwen3-32b`
Muguntha	NVIDIA Nemotron-3-Ultra-550B (consistent with her primary model)

Silent Fail

If analysis errors, `learnedPreference = null` — AI works normally with standard prompt. No user-facing impact.

Coverage

All 12 AI assistants confirmed:

File	Staff	Status
`muguntha.js`	Muguntha	✅
`members-api.js`	Sajeepan, Hetheesha, Sonya, Sukirtha, Kamsi, Theekshy, Thivajini	✅
`requirement.js`	Dilaksi, Jefri, Thasitha, Mahima	✅

Day-by-Day Example (Kamsi)

```
Day 1: No preference — standard prompt, standard task card
       Kamsi asks: "declining pages detail", "quick wins", "show all declining"
       → 3 user messages saved in kamsi_ai_chat

Day 2: New day detected →
       AI analyses: "Kamsi drills into declining pages first, wants quick wins in brief"
       → saved as role='preference'
       → injected into system prompt
       → task card now leads with full declining page detail + quick wins pre-listed

Day 5+: Task card is personalised to exactly what Kamsi always wants
        She types less — AI already surfaces what she'd ask
```

Last updated: 2026-08-25 Built by: Piranav (AIOS Architect) + Claude Sonnet 4.6